INTERNSHIP COMPLETION REPORT

MERN Stack Development Internship

SHOPSPHERE

Multi-Vendor Social E-Commerce Platform

Presented by

CTRL+ALT+WIN & FEMME FORCES

Reyhaan S – 312325205176
Sai Saran S – 312325205188
Mrityunjai G – 312325205132
Pasupathi G – 312325205157
Priyadharshini P – 312325205165
Niraimathi V – 312325205141

B.Tech. Information Technology, St. Joseph's College of Engineering, Chennai

Submitted to

Mr. Gokul E, HR Manager
TechPower Solutions
#05, 1st Floor, 100 Feet Road, Vadapalani, Chennai – 600 026

Internship period: 18 June 2026 – 15 September 2026

ABSTRACT

This report is presented by the teams Ctrl+Alt+Win and Femme Forces and describes the internship completed by their six members at TechPower Solutions, Chennai, from 18 June to 15 September 2026, in the domain of MERN stack development. The internship centred on the teams' joint project, ShopSphere, a multi-vendor social e-commerce platform.

ShopSphere consists of five React 19 single-page applications — a customer storefront, a seller portal, an administrator console, a delivery-partner console and a shared login application — served by one Node.js/Express 5 API with MongoDB persistence through eleven Mongoose models. The platform implements seller onboarding with GSTIN, PAN and document validation, administrator verification and product approval, cart and checkout with server-side pricing, PIN-code-based delivery assignment, a phased order journey, cancellations, returns, notifications over Socket.io and optional Razorpay test-mode payments with HMAC signature verification.

The report explains the system architecture, data model and functional modules, and examines the security design in detail: the shared login, role-aware credential checking with bcrypt and timing-normalised failures, seven-day JSON Web Tokens, the safe cross-origin session hand-off, and the composable Express middleware that enforces roles, ownership and a super-administrator tier on the server. The contribution of each team member is described, with main contributors distinguished from supporting members.

The system was run locally for verification: all six repository test scripts passed (107 checks), and a focused set of authentication and authorization checks matched the implemented behaviour. The verification also identified limitations — notably unauthenticated order and delivery-partner read endpoints and token storage in browser storage — which are recorded with recommended enhancements.

Keywords: MERN stack, multi-vendor e-commerce, JWT, role-based access control, Express middleware, MongoDB, React.

TABLE OF CONTENTS

ABSTRACT ..... **i**
LIST OF FIGURES AND TABLES ..... **iii**
1. INTRODUCTION AND COMPANY PROFILE **1**
1.1 Overview of the Internship 1
1.2 Company Profile 1
1.3 Objectives of the Internship 1
1.4 Scope and Organisation of the Report 1
2. PROJECT OVERVIEW AND REQUIREMENTS **3**
2.1 Problem Statement 3
2.2 Limitations Addressed and Proposed Solution 3
2.3 Joint Project Teams 3
2.4 Software Requirements and Development Environment 3
2.5 Technology Stack 4
3. SYSTEM DESIGN **5**
3.1 Overall Architecture 5
3.2 Frontend-to-Backend Request Flow 6
3.3 Database Design and Data Models 6
4. FUNCTIONAL MODULES AND WORKFLOWS **7**
4.1 Customer Storefront 7
4.2 Seller Portal 8
4.3 Administrator Console 8
4.4 Delivery Partner Console 8
4.5 Core Workflows 9
4.6 Payments, Notifications and Supporting Modules 10
5. AUTHENTICATION AND AUTHORIZATION IMPLEMENTATION **11**
5.1 Shared Login and Role Selection 11
5.2 Login Processing on the Server 11
5.3 Cross-Portal Session Hand-off 12
5.4 Authorization Middleware 13
5.5 Password, Reset and Registration Controls 14
6. TESTING AND VERIFICATION **15**
6.1 Test Environment and Approach 15
6.2 Repository Test Suites 15
6.3 Authentication and Authorization Test Cases 15
7. TEAM CONTRIBUTIONS AND COLLABORATION **17**
7.1 Team Structure 17
7.2 Member Contributions 17
7.3 Repository Evidence 18
7.4 Work Allocation Summary 18
8. CHALLENGES, OUTCOMES AND LIMITATIONS **20**
8.1 Challenges and Solutions 20
8.2 Project Outcomes 20
8.3 Limitations Identified 20
9. LEARNING OUTCOMES, CONCLUSION AND FUTURE ENHANCEMENTS **22**
9.1 Learning Outcomes 22
9.2 Conclusion 22
9.3 Future Enhancements 22
REFERENCES **23**

LIST OF FIGURES AND TABLES

Figures

Figure 3.1 Overall ShopSphere system architecture (drawn from the repository structure).....5
Figure 3.2 Principal collections and relationships (from backend/models/*.model.js).....6
Figure 4.1 Customer home page after sign-in.....7
Figure 4.2 Product details page.....7
Figure 4.3 Shopping cart with order summary.....7
Figure 4.4 Checkout with saved addresses.....7
Figure 4.5 My Orders inside the account area.....7
Figure 4.6 Order tracking timeline (ORD-1021).....7
Figure 4.7 Seller dashboard (TechNova Electronics demo store).....8
Figure 4.8 Seller product and stock management.....8
Figure 4.9 Seller verification queue with submitted documents.....8
Figure 4.10 Product approval queue.....8
Figure 4.11 Admin delivery assignment by PIN-code eligibility.....9
Figure 4.12 Delivery console for a signed-in partner.....9
Figure 4.13 Seller onboarding and product approval workflow.....9
Figure 4.14 Order lifecycle across customer, seller, admin and delivery roles.....9
Figure 5.1 Authentication flow from the shared login to a role portal.....12
Figure 5.2 Shared login: role chooser (port 5177).....12
Figure 5.3 Role-specific sign-in screen.....12
Figure 5.4 Generic error for an invalid password.....13
Figure 5.5 Unauthenticated admin visit redirected to admin sign-in.....13
Figure 5.6 Server-side authorization pipeline and rejection responses.....13

Tables

Table 2.1 Software requirements and development environment (from README.md and package.json files).....3
Table 2.2 Technology stack confirmed in the repository.....4
Table 5.1 Authorization guards and where they are applied.....13
Table 6.1 Results of the repository test scripts (backend/test, run on 21 October 2026).....15
Table 6.2 Authentication and authorization checks (expected vs. observed).....15
Table 7.1 Team members and responsibilities — Ctrl+Alt+Win and Femme Forces.....17
Table 7.2 Notable commits in the shared repository.....18
Table 7.3 Recurring task areas per member in the work-allocation calendar (days allocated in brackets). .19
Table 7.4 Project phases reconstructed from repository history and the allocation workbook.....19

CHAPTER 1
INTRODUCTION AND COMPANY PROFILE

1.1 Overview of the Internship

This report is presented jointly by the teams **Ctrl+Alt+Win** and **Femme Forces**, six students of B.Tech. Information Technology, St. Joseph's College of Engineering, Chennai, who completed an internship at **TechPower Solutions**, Chennai, from **18 June 2026 to 15 September 2026**. The internship was carried out in the MERN stack domain (MongoDB, Express.js, React and Node.js) and centred on a single team project: **ShopSphere — a Multi-Vendor Social E-Commerce Platform**.

ShopSphere connects four kinds of users — customers, sellers, platform administrators and delivery partners — through five React web applications that share one Node.js/Express API and one MongoDB database. The source code is maintained in the repository https://github.com/reyhaan07/E-Commerce [1]. The work was divided among the members by module, with three main contributors and three supporting members; the division of responsibilities is described in Chapter 7.

1.2 Company Profile

TechPower Solutions is an information technology company located at #05, 1st Floor, 100 Feet Road, Vadapalani, Chennai – 600 026, Tamil Nadu (website: www.techpowersolutions.in). The organisation works in software development, technical training, industrial internships and project-based learning, with the stated aim of bridging the gap between academic education and industry requirements. Its training areas include Full Stack and MERN Stack Development, Artificial Intelligence, Machine Learning, Data Analytics, Cloud Computing, the Internet of Things, Mobile Application Development, Python, Java and Software Testing.

The company follows a structured Software Development Life Cycle (requirement analysis, design, development, testing, deployment and maintenance) for its internship projects. Internship programmes are coordinated by **Mr. Gokul E, HR Manager**, who handles student onboarding, training schedules and professional guidance during the internship period.

1.3 Objectives of the Internship

- Gain practical experience in full-stack web development with the MERN stack across the complete development life cycle.
- Build RESTful APIs with Node.js and Express, and model persistent data in MongoDB through Mongoose.
- Design and implement secure authentication and role-based authorization for a multi-role application using JSON Web Tokens (JWT) and bcrypt password hashing.
- Integrate several independently deployed React front-ends with a shared backend, and debug cross-application issues.
- Practise collaborative development with Git and GitHub, API testing, regression checking and technical documentation.

1.4 Scope and Organisation of the Report

The technical content of this report was verified against the repository's main branch (commit `8f85bbd`, inspected on 21 October 2026). The application was installed and run

locally for this report, its automated test scripts were executed, and every screenshot was captured from that running instance (Section 6.1). Chapter 2 introduces the project and its technology; Chapter 3 presents the architecture and data model; Chapter 4 the functional modules; Chapter 5 the authentication and authorization implementation; Chapter 6 testing; Chapter 7 the team's contributions and work allocation; Chapter 8 challenges, outcomes and limitations; and Chapter 9 learning outcomes, conclusions and future enhancements.

CHAPTER 2
PROJECT OVERVIEW AND REQUIREMENTS

2.1 Problem Statement

A marketplace with many independent sellers has to coordinate several parties around a single order: the customer who pays, the seller who packs it, an operator who approves sellers and listings and assigns couriers, and the courier who delivers it. Each party needs its own interface and must see only the data and actions appropriate to its role. ShopSphere was built to provide these role-specific interfaces over one consistent data store, with secure sign-in and enforced permissions as a core requirement rather than an afterthought.

2.2 Limitations Addressed and Proposed Solution

The repository history shows how the design evolved. In its early form (June–July 2026) each front-end carried its own login page and the backend kept orders and delivery partners in an in-memory, JSON-persisted store. The commit of 16 July 2026, *"Migrate backend to MongoDB and consolidate login into a shared app"*, replaced these with a MongoDB database and a single shared login application. The resulting solution has the following characteristics:

- **One identity service for four roles.** A shared login app authenticates every role against `POST /api/login` and routes the user to the correct portal.
- **Server-side enforcement.** Every privileged API route is guarded by composable middleware (`requireAuth`, `requireRole`, `requireSelfOrAdmin`, `requireSuperAdmin`), so hiding a page in the UI is never the only protection.
- **Persistent, validated data.** Eleven Mongoose models with enumerated statuses and an explicit seller status-transition table prevent invalid order states.
- **Coordinated workflows.** Seller verification, product approval, PIN-code-based courier assignment, delivery tracking, cancellations and returns are connected end to end, with real-time Socket.io notifications.

2.3 Joint Project Teams

ShopSphere was developed as a joint project of two teams, **Ctrl+Alt+Win** and **Femme Forces**, comprising six students. The division of responsibilities, as recorded in the team's work-allocation workbook, is summarised in Section 7.3.

2.4 Software Requirements and Development Environment

Table 2.1: Software requirements and development environment (from README.md and package.json files)

Item	Requirement / version used
Runtime	Node.js v20 or higher (verified on v22.22.0)
Database	MongoDB (local service at mongodb://127.0.0.1:27017/shopsphere, or a remote URI via backend/.env); verified with MongoDB 7
Package manager / scripts	npm; root scripts `install:all`, `dev:all` (concurrently runs 6 processes) and `build:all`
Backend configuration	backend/.env: PORT, MONGO*URI*, *JWT*SECRET, LOGIN*APP*URL; optional SMTP_* and Razorpay test keys
Development ports	API 5000; Admin 5173; Seller 5174; Customer 5175; Delivery 5176; Shared login 5177

Item	Requirement / version used
Tools	Visual Studio Code, Git and GitHub, a web browser with developer tools, API test scripts in backend/test
Client	Any modern browser; each portal is a single-page application served by Vite

2.5 Technology Stack

Table 2.2: Technology stack confirmed in the repository

Layer	Technology	Use in ShopSphere
Frontend	React 19, Vite 8, React Router 7	Five single-page applications (customer, seller, admin, delivery, shared login)
Styling / UI	Tailwind CSS 4, React Icons, Framer Motion	Shared black-and-blue design system; plain CSS in the delivery console
State / charts	Redux Toolkit, Axios, Recharts (seller portal)	Seller state store, HTTP client and analytics charts
Backend	Node.js, Express 5	REST API with 13 route modules under /api
Database	MongoDB, Mongoose 9	11 schemas with validation, indexes and toJSON transforms
Security	jsonwebtoken 9, bcryptjs 3	JWT issue/verify (7-day expiry); salted password hashing (cost 10)
Real time	Socket.io 4 (server and client)	Role and user rooms for order and notification events
Integrations	Razorpay SDK (test mode), Nodemailer	Optional online payment with HMAC signature verification; optional SMTP e-mail with console fallback
Version control	Git, GitHub	Shared repository; pull-request based merges

The commit history also records co-author trailers from an AI coding assistant on many commits, across several members’ accounts; AI-assisted development was therefore part of the team’s workflow.

CHAPTER 3
SYSTEM DESIGN

3.1 Overall Architecture

ShopSphere follows a three-tier architecture (Figure 3.1). The presentation tier consists of five React applications [8] built with Vite [9], each served on its own development port and therefore on its own browser origin. All of them call the same backend over REST/JSON and open a Socket.io connection for live updates. The application tier is a single Express 5 server [5] (`backend/server.js`) that mounts thirteen route modules, applies authorization middleware per route and converts every unhandled error into a uniform `{ success: false, message }` JSON response. The data tier is a MongoDB database [7] accessed only through Mongoose models [6]. Razorpay and SMTP are optional: without credentials, checkout falls back to Cash on Delivery and e-mails are written to the server console.

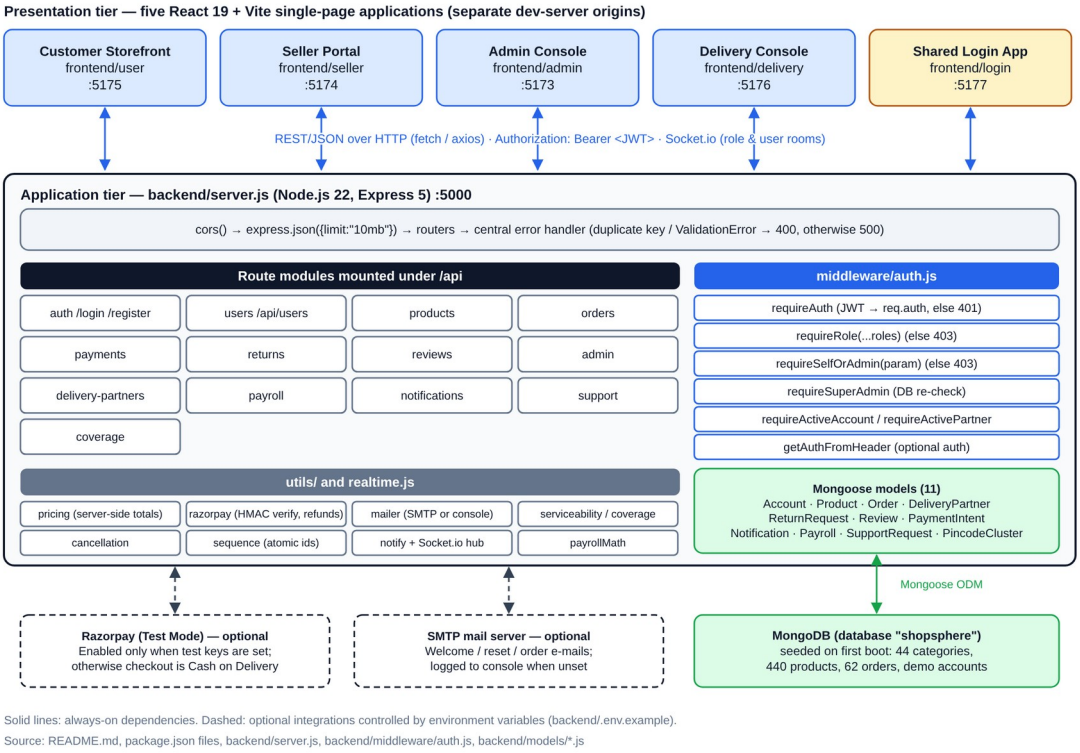


Figure 3.1: Overall ShopSphere system architecture (drawn from the repository structure)

3.2 Frontend-to-Backend Request Flow

Every portal has a small API client. The customer storefront's `apiRequest()` (frontend/user/src/api/client.js) reads the saved session from `localStorage`, attaches `Authorization: Bearer <token>`, parses the JSON response and turns `success: false` into a thrown error that the page displays. A `401` response clears the stale session and redirects to the login with a `redirect` parameter so the user returns to the same page after signing in. On the server, a request passes through `cors()` and the JSON body parser, reaches its router, runs the route's middleware chain (Figure 5.6) and then the handler wrapped in `asyncHandler`, which forwards rejected promises to the central error handler. Mongoose duplicate-key errors become `400 "That value is already in use"`, validation errors become `400`, and anything else becomes a generic `500` so internal details are not leaked.

Real-time updates use a Socket.io hub [10] (`backend/realtime.js`). Clients connect with a `role` and optional `userId` query parameter and are joined to rooms such as `role:admin` or `user:user-1`; route handlers call `emitToRole` and `emitToUser` after state changes, for example when an order is placed or a delivery status changes.

3.3 Database Design and Data Models

The database uses eleven Mongoose models (Figure 3.2). A single **Account** collection stores customers, sellers and administrators, distinguished by `role`; delivery partners are kept in a separate **DeliveryPartner** collection with their own account status, vehicle and document fields. Every model uses a readable business key (`id`, such as "user-1", "prod-59" or "ORD-1021") that the API and front-ends reference, generated by an atomic counter (`utils/sequence.js`). The `toJSON` transforms strip `_id`, `__v`, password hashes and reset tokens from every API response. Orders embed snapshots of item name, price and image so that order history survives a product being delisted, and record every state change in `statusHistory` with its phase, actor and timestamp.

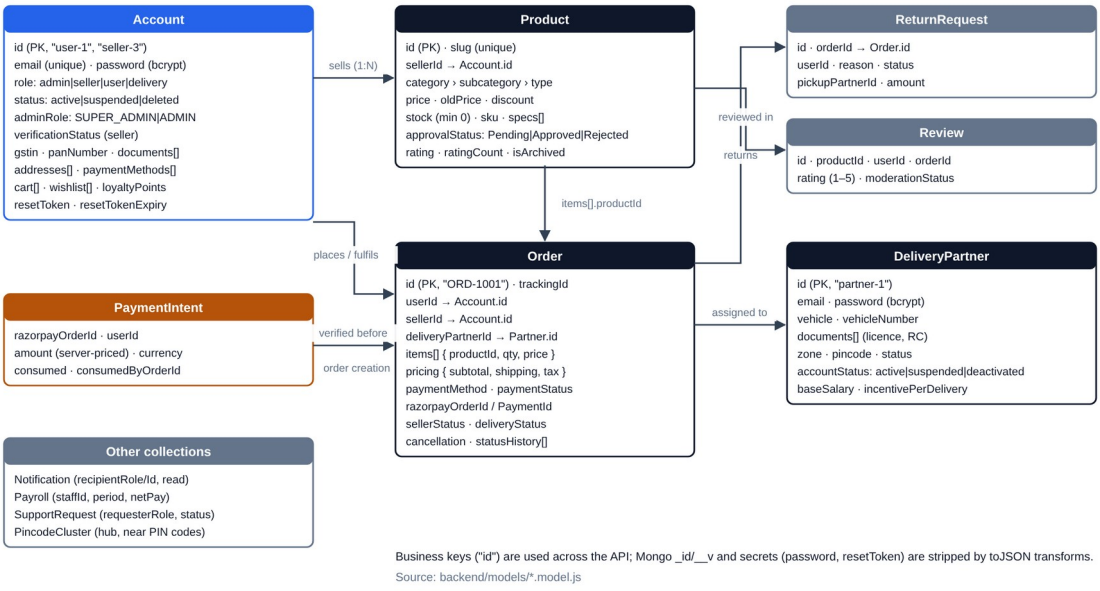


Figure 3.2: Principal collections and relationships (from backend/models/*.model.js)

CHAPTER 4
FUNCTIONAL MODULES AND WORKFLOWS

This chapter describes the modules that were confirmed in the code and exercised in the running application. The screenshots were captured on 21 October 2026 from the current main branch using the seeded demo data (no real customer data). Product photographs are loaded from an external image CDN that was not reachable from the capture environment, so image areas show placeholder or alternative text.

4.1 Customer Storefront

The storefront (frontend/user, port 5175) provides a home page with category navigation, a filterable product listing (category hierarchy, price, rating and sort), product details with specifications, a cart, a wishlist and a checkout page (Figures 4.1–4.4). Only products whose `approvalStatus` is “Approved” and which are not archived are returned to customers. Checkout calls `POST /api/orders`; the server prices the cart from the catalogue rather than trusting the browser, checks PIN-code serviceability, decrements stock, notifies the seller and administrators in real time and sends a confirmation e-mail. The account area lists the customer’s orders and links to a tracking page that renders the order journey (Figures 4.5 and 4.6). Checkout, profile, wishlist and tracking routes are wrapped in a `RequireAuth` component.

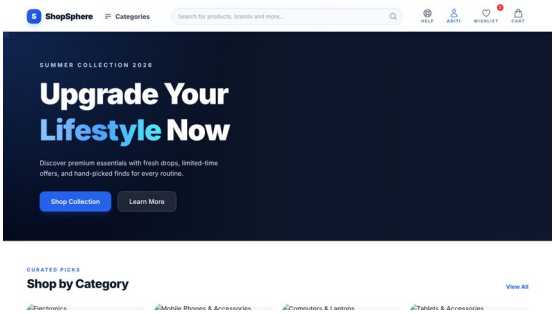


Figure 4.1: Customer home page after sign-in

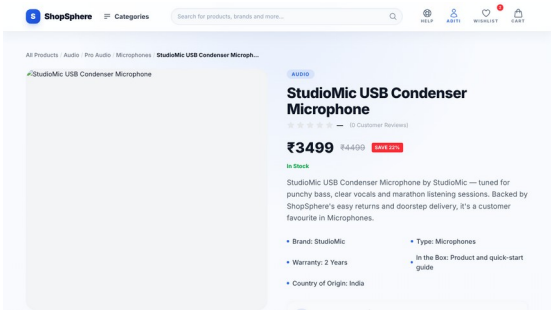


Figure 4.2: Product details page

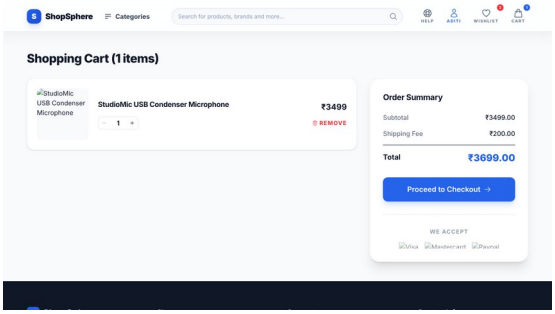


Figure 4.3: Shopping cart with order summary

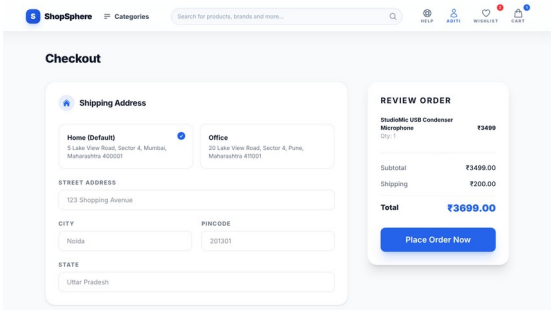


Figure 4.4: Checkout with saved addresses

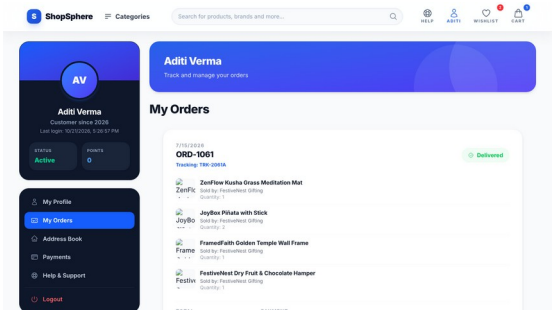


Figure 4.5: My Orders inside the account area

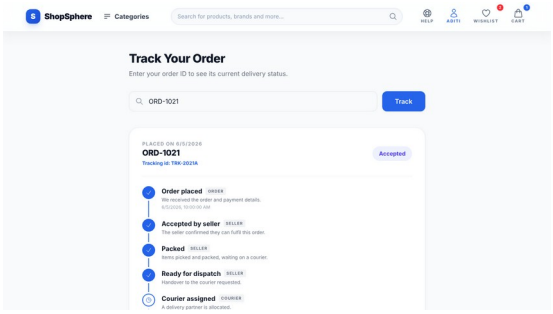


Figure 4.6: Order tracking timeline (ORD-1021)

4.2 Seller Portal

The seller portal (frontend/seller, port 5174) offers a dashboard with revenue and order statistics, product management, inventory steppers, order processing, analytics, discounts, payouts and store settings including serviceable PIN codes (Figures 4.7 and 4.8). Sellers register through `POST /api/register/seller`, which validates the phone number, the 15-character GSTIN and 10-character PAN formats and the four mandatory documents, and creates the account with `verificationStatus: "Pending"`. A seller can publish products only after an administrator verifies the store; each new listing then enters the approval queue (Figure 4.13). Seller order updates are validated against the `SELLER_TRANSITIONS` table, so an order cannot jump from "Processing" to "Delivered".

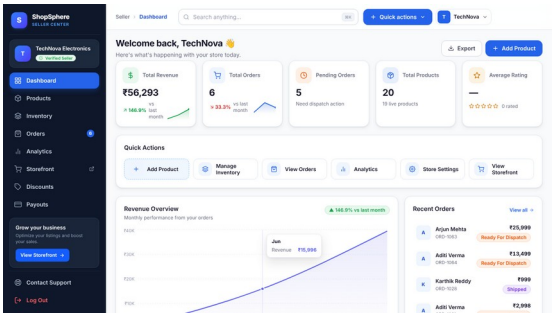


Figure 4.7: Seller dashboard (TechNova Electronics demo store)

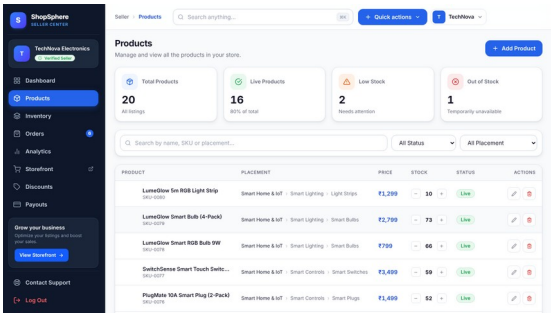


Figure 4.8: Seller product and stock management

4.3 Administrator Console

The admin console (frontend/admin, port 5173) contains the dashboard, user, seller and product management, seller verification with document review, product approval, order monitoring, refunds, delivery-partner management, delivery assignment and tracking, PIN-code coverage, payroll, support inquiries and, for super administrators only, the admin roster (Figures 4.9–4.11). The whole `/api/admin` router is protected by `requireAuth` and `requireRole("admin")`; the roster and payroll routes additionally require the SUPER_ADMIN tier.

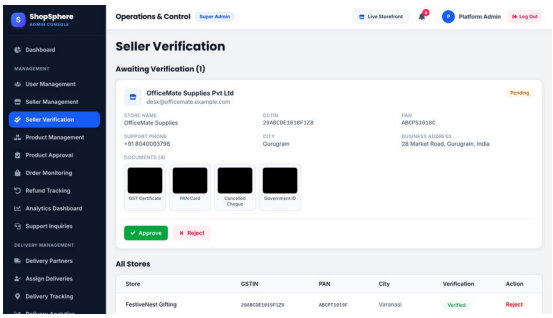


Figure 4.9: Seller verification queue with submitted documents

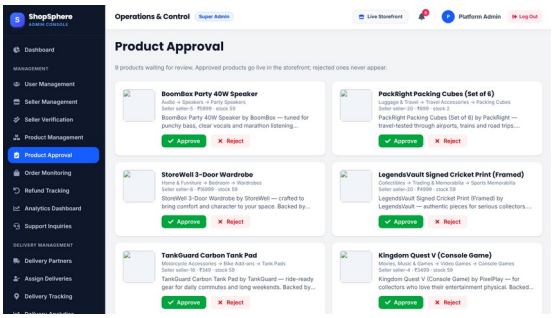


Figure 4.10: Product approval queue

4.4 Delivery Partner Console

Delivery partners use frontend/delivery (port 5176) to view assigned orders, advance each job through Accepted, Picked Up, In Transit, Out For Delivery and Delivered, handle reverse pickups for returns and review their delivery history (Figure 4.12). The administrator assigns a partner from a list of eligible partners matched by PIN-code catchment (Figure 4.11). The update endpoint checks that the partner owns the job before changing it.

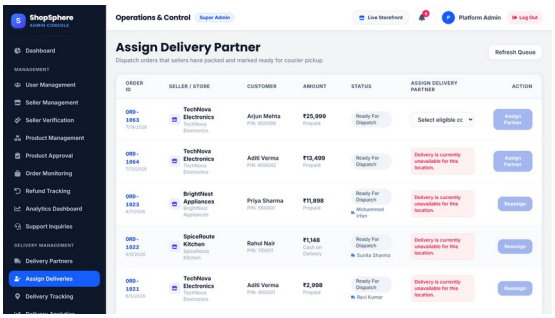


Figure 4.11: Admin delivery assignment by PIN-code eligibility

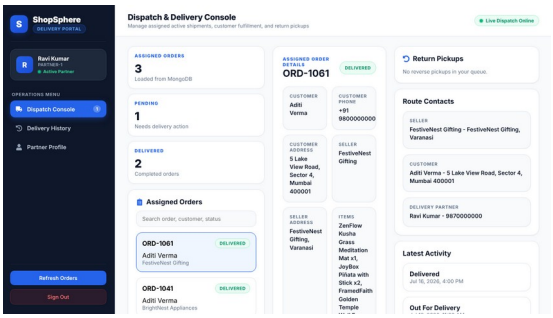
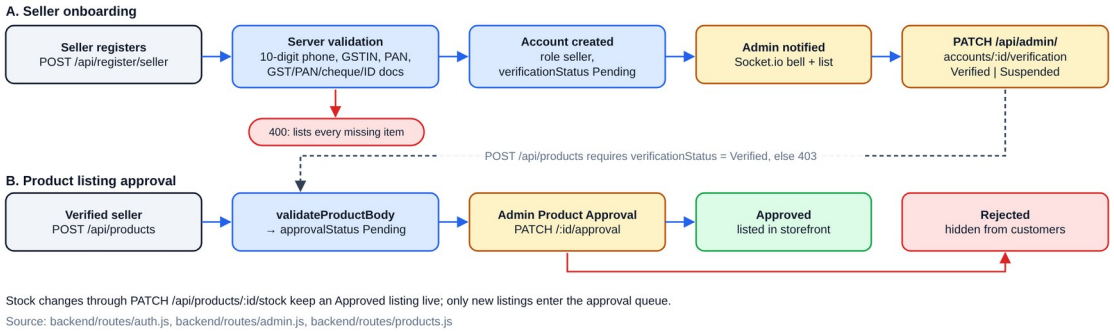


Figure 4.12: Delivery console for a signed-in partner

4.5 Core Workflows

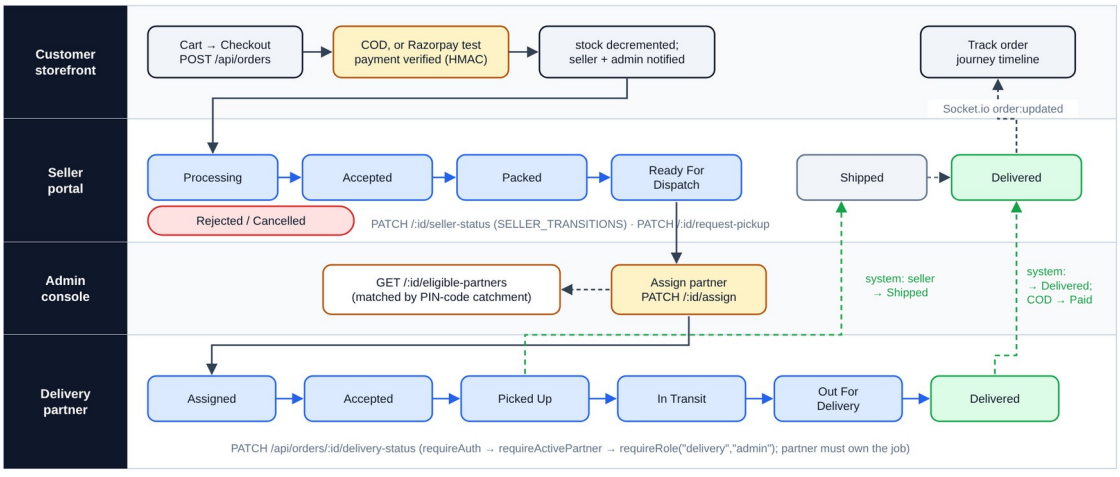
Figure 4.13 shows the seller onboarding and product approval workflow, and Figure 4.14 the complete order lifecycle across the four roles. When a courier marks an order “Picked Up”, the server automatically moves the seller status to “Shipped”; on “Delivered” it marks the seller status delivered and, for Cash on Delivery, records the payment as paid. Cancellations before dispatch restore stock and record refunds, and returns follow a separate pipeline from request through reverse pickup and inspection to refund (routes/returns.js).



Stock changes through PATCH /api/products/:id/stock keep an Approved listing live; only new listings enter the approval queue.

Source: backend/routes/auth.js, backend/routes/admin.js, backend/routes/products.js

Figure 4.13: Seller onboarding and product approval workflow



Each hop is appended to order.statusHistory {status, phase, actor, note, timestamp}; buildJourney() folds it into the timeline shown to the customer and seller.

Cancellation (POST /:id/cancel, item-level cancel) and returns (Requested → … → Refund Approved, routes/returns.js) branch off this main path.

Source: backend/models/order.model.js (SELLER_STATUSES, SELLER_TRANSITIONS, DELIVERY_STATUSES), backend/routes/orders.js

Figure 4.14: Order lifecycle across customer, seller, admin and delivery roles

4.6 Payments, Notifications and Supporting Modules

Online payment uses Razorpay in test mode only when test keys are configured. `POST /api/payments/create-order` prices the cart on the server and stores a **PaymentIntent**; order creation then verifies the Razorpay signature with HMAC-SHA256 and a constant-time comparison (`crypto.timingSafeEqual`) as described in Razorpay's documentation [11], and checks that the intent's amount matches the order total, so a payment for a smaller amount cannot be reused for a larger cart. In the environment used for this report no Razorpay keys were configured; the payment test suite confirmed that online payment is refused and Cash on Delivery works. **No live or test online payment was performed for this report.** Other implemented modules include in-app notifications, product reviews with moderation, support requests and payroll generation for delivery partners.

CHAPTER 5
AUTHENTICATION AND AUTHORIZATION IMPLEMENTATION

Within the team, the authentication and authorization work was led by Reyhaan S. This chapter describes the mechanism as it exists on the current main branch. Items marked † were added in the post-internship commit `5452ea8` (20 October 2026) and are described for completeness only; Section 7.3 lists the related commits.

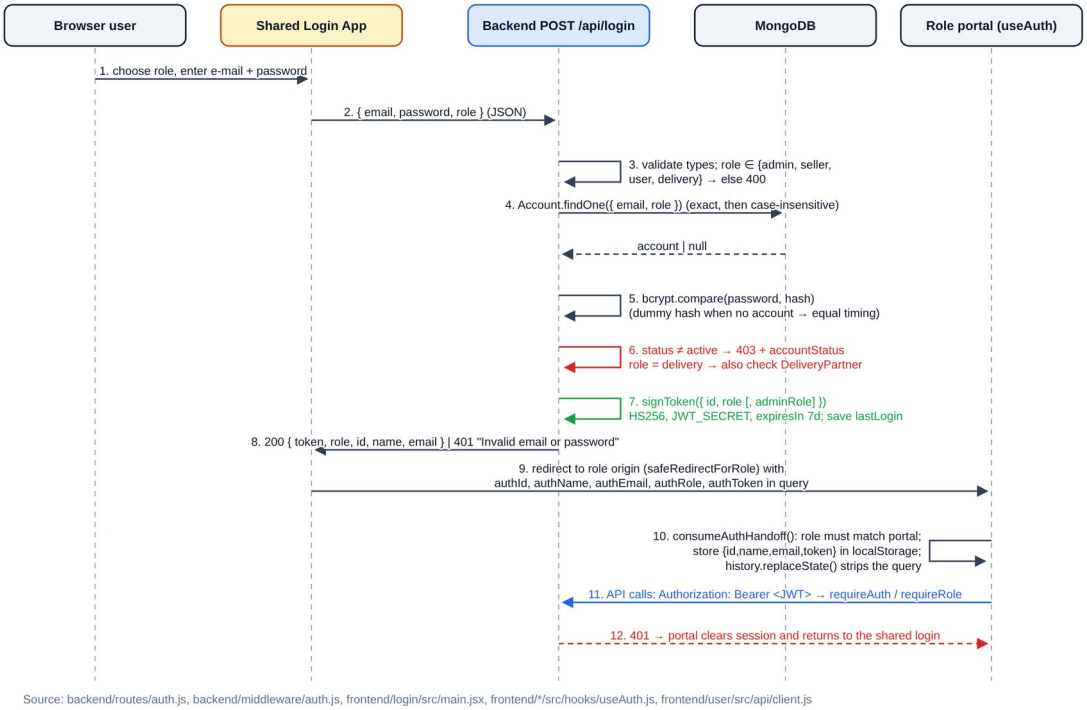
5.1 Shared Login and Role Selection

The shared login application (frontend/login, port 5177) is the single sign-in entry point for all four roles. The user first chooses a role (Figure 5.2) and then signs in on a role-specific screen reached through `/login/<role>` or the `?role=` deep link used by the other portals (Figure 5.3); a wrong password produces the generic error in Figure 5.4. The same app hosts the forgot-password and reset-password forms and links to each role's registration page.

5.2 Login Processing on the Server

`POST /api/login` (backend/routes/auth.js) performs the following steps (Figure 5.1):

1. **Input validation.** `email`, `password` and `role` must be strings and `role` must be one of `admin`, `seller`, `user` or `delivery`; otherwise the server returns `400`.
2. **Account lookup.** The account is looked up by e-mail *and* role, first exactly and then case-insensitively with regular-expression metacharacters escaped. The same e-mail used for a customer account therefore cannot sign in to the seller portal.
3. **Password verification.** `bcrypt.compare` checks the password against the stored hash. When no account exists, the server still compares against a dummy hash so that "unknown e-mail" and "wrong password" take similar time and return the same `401 "Invalid email or password"` message, which limits account enumeration, as recommended by OWASP [12].
4. **Account standing.** A suspended or deleted account is refused with `403` and an `accountStatus` field, which the login screen shows as a dedicated panel instead of a credentials error†. For the delivery role, partners stored in the DeliveryPartner collection are also checked.
5. **Token issue.** `signToken()` creates a JWT containing only `id` and `role` (plus `adminRole` for administrators), signed with `JWT_SECRET` (HMAC-SHA256, the library default [3]) and valid for seven days, following the JWT format of RFC 7519 [2]; `lastLogin` is updated and the response returns the token with the user's id, name and e-mail.



Source: backend/routes/auth.js, backend/middleware/auth.js, frontend/login/src/main.jsx, frontend/*/src/hooks/useAuth.js, frontend/user/src/api/client.js

Figure 5.1: Authentication flow from the shared login to a role portal

5.3 Cross-Portal Session Hand-off

Because each portal runs on a different origin, `localStorage` cannot be shared between the login app and the portals. After a successful login the login app therefore builds the destination URL with `safeRedirectForRole()`, which accepts a `redirect` parameter only if its origin is the selected role's own portal (preventing an open redirect to another site), and appends `authId`, `authName`, `authEmail`, `authRole` and `authToken` as query parameters. In the portal, `consumeAuthHandoff()` in the `useAuth` hook accepts the hand-off only when `authRole` matches the portal's expected role, saves the session under a portal-specific key, and immediately removes the parameters from the address bar with `history.replaceState()`. The seller and admin portals protect their layouts with a `RequireAuth` component that performs a hard redirect to the shared login (`?role=seller` or `?role=admin` with a `redirect` back), as Figure 5.5 shows for an unauthenticated visit to the admin dashboard.

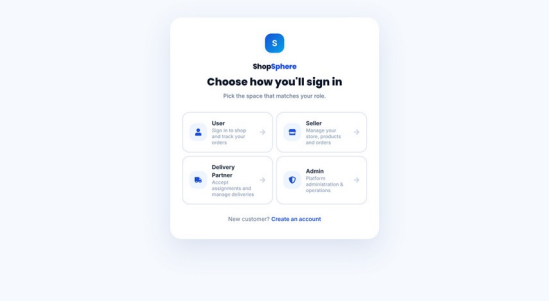


Figure 5.2: Shared login: role chooser (port 5177)

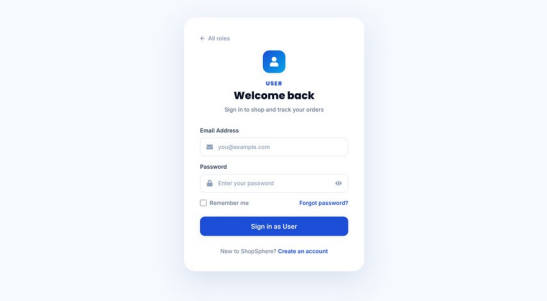


Figure 5.3: Role-specific sign-in screen

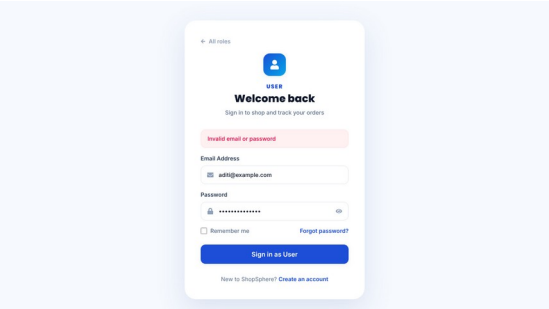


Figure 5.4: Generic error for an invalid password

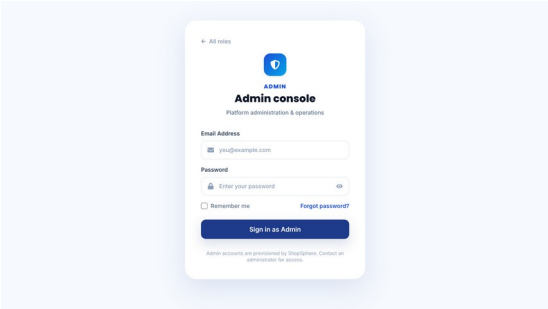
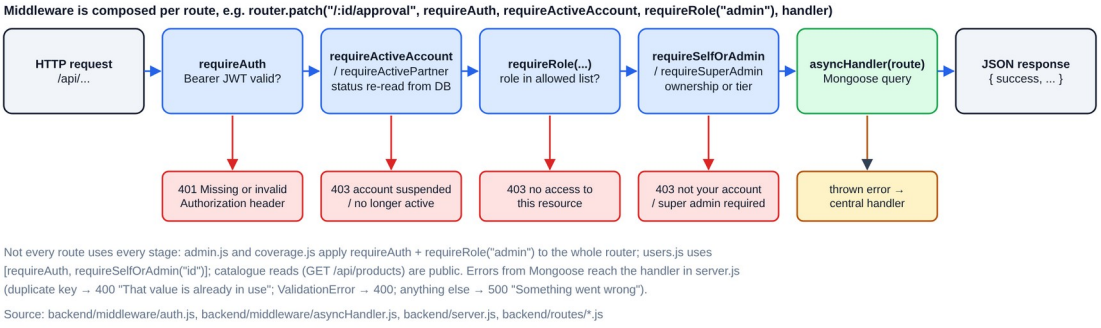


Figure 5.5: Unauthenticated admin visit redirected to admin sign-in

5.4 Authorization Middleware

Authorization is enforced on the server by small, composable middleware functions in `backend/middleware/auth.js`, applied per route or to a whole router (Figure 5.6 and Table 5.1). The essential pattern is shown below.

```
function requireAuth(req, res, next) {
  const auth = getAuthFromHeader(req);          // verifies "Bearer <jwt>" with
JWT_SECRET
  if (!auth) return res.status(401).json({ success: false,
                    message: "Missing or invalid Authorization header" });
  req.auth = auth;                              // { id, role [, adminRole] }
  next();
}
function requireRole(...roles) {
  return (req, res, next) => roles.includes(req.auth.role) ? next()
    : res.status(403).json({ success: false, message: "You don't have access to
this resource" });
}
// usage: router.patch("/:id/approval", requireAuth, requireActiveAccount,
requireRole("admin"), handler)
```



Not every route uses every stage: admin.js and coverage.js apply requireAuth + requireRole("admin") to the whole router; users.js uses [requireAuth, requireSelfOrAdmin("id")]; catalogue reads (GET /api/products) are public. Errors from Mongoose reach the handler in server.js (duplicate key → 400 "That value is already in use"; ValidationError → 400; anything else → 500 "Something went wrong").

Source: backend/middleware/auth.js, backend/middleware/asyncHandler.js, backend/server.js, backend/routes/*.js

Figure 5.6: Server-side authorization pipeline and rejection responses

Table 5.1: Authorization guards and where they are applied

Guard	Behaviour	Examples of use
requireAuth	Rejects with 401 unless a valid Bearer JWT is present; sets req.auth	All private routes
requireRole(...)	403 unless req.auth.role is in the allowed list	admin and coverage routers; product create/approve; seller-status; assignment

Guard	Behaviour	Examples of use
requireSelfOrAdmin(param)	403 unless the caller owns the :id or is an admin	Every /api/users/:id route (profile, addresses, cart, wishlist)
requireSuperAdmin	Re-reads the caller from the database and requires an active SUPER_ADMIN†	Admin roster, payroll generation and edits
requireActiveAccount / requireActivePartner†	Re-reads account status so a suspension takes effect before the 7-day token expires	Order, product, payment and delivery updates
getAuthFromHeader	Optional authentication without failing the request	GET /api/orders?userId= (owner or admin only)
Frontend RequireAuth	Redirects unauthenticated users to the login; hides super-admin screens	Checkout, profile, seller layout, admin layout

5.5 Password, Reset and Registration Controls

- Passwords are hashed with bcrypt [4] (cost factor 10) in a Mongoose `pre("save")` hook, so every code path that sets a password — registration, seed data, admin reset or password reset — stores only a hash. `npm run hash-passwords` re-hashes any legacy plaintext password.
- Registration enforces a valid e-mail format, a minimum password length of eight characters and a 10-digit phone number, and the unique index on e-mail blocks duplicates even under concurrent requests.
- `POST /api/forgot-password` returns the same message whether or not the account exists; the reset token is 20 random bytes, expires after one hour and is cleared after use.
- The seeded founding administrator cannot be demoted, suspended or deleted, so the console cannot be locked out of its own roster.

CHAPTER 6
TESTING AND VERIFICATION

6.1 Test Environment and Approach

For this report the repository was cloned at commit `8f85bbd`, dependencies were installed with `npm install` and `npm run install:all`, MongoDB 7 was run in a local Docker container and all six processes were started with `npm run dev:all` using a locally generated `JWT_SECRET`. Testing combined (a) the backend test scripts that ship with the repository, which call the running API and print PASS or FAIL for each check, (b) a focused authentication and authorization check written for this report, and (c) manual walkthroughs of each portal while capturing the screenshots with Playwright [14]. All tests were run on 21 October 2026, after the internship period; they verify the current code, not historical behaviour.

6.2 Repository Test Suites

Table 6.1: Results of the repository test scripts (backend/test, run on 21 October 2026)

Script	What it checks	Result
smoke.js (npm test)	Catalogue loads; login for all four roles; registration	6 passed, 0 failed
features.js	Order journey, status transitions and related features	24 passed, 0 failed
serviceability.test.js	PIN-code delivery serviceability rules	17 passed, 0 failed
cancellation.js	Order and item cancellation, stock restoration, refunds	39 passed, 0 failed
payments.js	Payment integrity in Cash-on-Delivery-only mode (no Razorpay keys)	6 passed, 0 failed
verify*changes*1*to*10.js	Regression checks for ten earlier change requests	15 passed, 0 failed

The payment script reports that prepaid paths are exercised only when the server is restarted with `ENABLE_MOCK_PAYMENTS=true`; that mode was not run, so prepaid checkout is **not** claimed as tested here. There are no automated front-end tests in the repository.

6.3 Authentication and Authorization Test Cases

Table 6.2 lists the security-related checks executed against the running API with the seeded demo accounts. Each row records the expected and the observed HTTP status.

Table 6.2: Authentication and authorization checks (expected vs. observed)

ID	Scenario	Expected	Observed
A1	Valid customer login	200 + JWT	200 ✔️
A2	Wrong password	401	401 ✔️
A3	Unknown e-mail (same message as A2)	401	401 ✔️
A4	Valid customer credentials used for the seller role	401	401 ✔️
A5–A6	Missing role / invalid role value	400	400 ✔️
B1–B2	GET /users/me with no token / tampered token	401	401 ✔️
C1–C2	Customer or seller calls GET /admin/stats	403	403 ✔️
C3	Admin calls GET /admin/stats	200	200 ✔️
C4–C5	Customer approves or creates a product	403	403 ✔️

ID	Scenario	Expected	Observed
C6	Delivery partner changes seller status	403	403 ✔️
D1	Customer reads another user’s profile	403	403 ✔️
D3–D4	Customer lists another user’s orders / no token	403 / 401	403 / 401 ✔️
D5	Admin reads any user profile	200	200 ✔️
F1–F3	Short password, malformed e-mail, invalid reset token	400	400 ✔️
G1	Password hash absent from /users/me response	absent	absent ✔️
G2–G3	JWT payload fields and lifetime	id, role; 7 days	id, role, iat, exp; 7 days ✔️
H1–H2	GET /orders and GET /orders/:id without a token	(not restricted in code)	200 — limitation, see 8.3

CHAPTER 7
TEAM CONTRIBUTIONS AND COLLABORATION

7.1 Team Structure

ShopSphere was developed jointly by the teams **Ctrl+Alt+Win** and **Femme Forces**. Work was divided by module: three members acted as main contributors with ownership of a core area, one member supported the seller module, and two members carried out limited, bounded testing and evidence tasks. Table 7.1 summarises the responsibilities recorded in the team's Task Assignment sheet.

Table 7.1: Team members and responsibilities — Ctrl+Alt+Win and Femme Forces

Member (Reg. No.)	Level	Main responsibility
Reyhaan S (312325205176)	Main contributor	Authentication, authorization, shared login, protected routes, security validation and integration
Sai Saran S (312325205188)	Main contributor	System architecture, frontend/backend integration, API and data flow, build configuration and coordination
Mrityunjai G (312325205132)	Main contributor	Administration portal, seller and product approvals, role permissions, order and delivery administration
Pasupathi G (312325205157)	Supporting contributor	Seller portal, catalogue and inventory workflows, seller fulfilment and testing
Priyadharshini P (312325205165)	Limited support	Customer UI spot-checks, cart/checkout smoke checks, screenshot evidence
Niraimathi V (312325205141)	Light support	Occasional delivery-workflow checks, screenshots and simple QA notes

7.2 Member Contributions

- **Reyhaan S — authentication, authorization and integration.** The shared login application and per-role login entry, the safe redirect to each role's portal, the super-administrator tier (`requireSuperAdmin` and `adminRole` in the token), removal of the e-mail OTP step with a migration that re-hashes plaintext passwords, storefront re-authentication on `401`, the four-role login smoke test, and security and cross-portal regression checks (Chapter 5).
- **Sai Saran S — architecture, integration and coordination.** Overall system architecture and the mapping of each portal to its backend routes, frontend/backend data flow (including cart-to-order integration), build and run configuration, cross-portal API consistency, error-handling alignment, integration test planning and documentation; the later UI and data-flow enhancements merged as pull request #3 were committed under the Sai Saran S account.
- **Mrityunjai G — administration portal.** Seller verification and product approval workflows, user and seller management, role-permission checks on admin-only screens and actions, order administration and delivery assignment, and status-transition verification (Sections 4.3 and 4.5).
- **Pasupathi G — seller portal (supporting).** Seller dashboard navigation, product-entry and validation review, inventory and stock workflows, seller-side order handling and fulfilment status, and consistency checks between seller listings and the storefront (Section 4.2).

- **Priyadharshini P — customer storefront checks (limited).** Cart and checkout smoke tests, checkout validation, customer UI spot-checks and organisation of customer-flow screenshots.
- **Niraimathi V — delivery checks (light).** Occasional delivery-status and navigation spot-checks, cross-role status-label comparison, simple form checks and help with labelling report evidence.

7.3 Repository Evidence

Table 7.2 lists notable commits from the internship period and the one later merge. Commit authorship is useful evidence but is not a complete record of who did what: much of the work was discussed and tested jointly, some members contributed through testing rather than commits, and several commits carry AI co-author trailers.

Table 7.2: Notable commits in the shared repository

Date	Commit (author)	Change
16 Jul 2026	631eed3 (Reyhaan S)	Migrated the backend to MongoDB; replaced each app's own login with the shared frontend/login app
17–18 Jul 2026	several (AI-assistant identity)	Catalogue, reviews, returns, real-time events, payments; storefront, seller, admin and delivery screens wired to the live API; seed data
19 Jul 2026	3384e7b (reyhaan07)	Payroll module, editable admin/seller profiles, per-role login entry, seller onboarding, delivery history
25 Jul 2026	dc43a8f, 998db43 (reyhaan07)	Removed e-mail OTP verification; password re-hash migration; backend smoke test
20 Aug 2026	e83285c (reyhaan07)	Super-admin roster behind requireSuperAdmin; order status-transition graph and phased journey; features test
20–22 Aug 2026	b00380b, 612e7f0, 44ba684, 4c808b0	Seller Center redesign and PIN-code delivery settings; customer profile redesign; storefront re-authentication on 401
20 Oct 2026	5452ea8 (Sai Saran S), merged as PR #3	UI and data-flow enhancements, account-status re-checks, delivery self-registration — after the internship period

Earlier commits (26 June – 13 July 2026) by the accounts "Blueundead24" and "Harini-G-07" created the initial project structure, the first login API, the delivery app and the user-account module, including the original `requireAuth` and `requireSelfOrAdmin` guards; these accounts are not mapped to team members in the allocation workbook.

7.4 Work Allocation Summary

The team's workbook covers **15 July 2026 to 14 September 2026**, a sub-period of the internship (18 June – 15 September 2026). It allocates one recurring task per member for each of 41 working days. The workbook itself states that its daily calendar is a proposed retrospective allocation and not a verified record of attendance or completion dates; it is summarised here as a work-allocation record, not as a daily log.

Table 7.3: Recurring task areas per member in the work-allocation calendar (days allocated in brackets)

Member	Recurring task areas
Reyhaan S	Role authorization (5), API auth integration (5), protected routes (4), auth integration fixes (4), security regression (4), environment configuration (4), cross-portal verification (4), credential validation (4), token/session handling (4), final auth walkthrough (3)
Sai Saran S	Build/configuration review (5), integration test planning (5), error-handling alignment, documentation consolidation, final integration review, scope and milestones, architecture review, cart/order integration, cross-portal API checks (4 each), API/data-flow tracing (3)
Mrityunjai G	Regression pass (5), evidence preparation (5), issue verification, seller approval workflow, role permission checks, product administration, user management, status-transition checks, admin API review (4 each), order administration (3)
Pasupathi G	Seller portal orientation (5), product creation review (5), seller order handling, catalogue consistency, fulfilment status, seller validation QA, evidence capture, cross-role check, final walkthrough (4 each), seller API trace (3)
Priyadharshini P	Checkout validation (5), customer screenshots (5), cart smoke test and other customer UI spot-checks (3–4 each)
Niraimathi V	Cross-role display check (5), light regression support (5), delivery UI and journey spot-checks, report evidence and handover support (3–4 each)

Table 7.4: Project phases reconstructed from repository history and the allocation workbook

Period	Activity	Evidence
18 Jun – 14 Jul	Onboarding and MERN training; initial project set-up, common login, first backend APIs and user-account module	Commits 26 Jun – 13 Jul
15 – 31 Jul	MongoDB migration and shared login; full module build-out; per-role login; OTP removal and password hashing; smoke test	Commits 16 – 25 Jul; workbook
1 – 31 Aug	Super-admin tier and order journey; seller centre redesign; PIN-code delivery settings; 401 re-authentication	Commits 20 – 22 Aug; workbook
1 – 15 Sep	Regression checks, cross-portal verification, documentation and report preparation	Workbook only (no commits dated in September)
After 15 Sep	UI and data-flow enhancements merged as pull request #3 on 20 Oct 2026	Commit 5452ea8 / 8f85bbd — outside the internship period

CHAPTER 8
CHALLENGES, OUTCOMES AND LIMITATIONS

8.1 Challenges and Solutions

The challenges below are those that are visible in the code and commit history.

- **Sharing a login across separate origins.** Each portal runs on its own port, so a session stored by the login app is invisible to the others. The solution was a one-time hand-off through redirect query parameters, accepted only for the matching role, stored per portal and immediately removed from the URL; redirect targets are restricted to the selected role's own origin.
- **Distinguishing roles for the same person.** Looking up accounts by e-mail *and* role keeps a customer login from opening the seller or admin console, and gives each portal a clear expected role.
- **Registration friction versus security.** The six-digit e-mail OTP step was removed from registration (commit dc43a8f); to keep credential storage safe, a migration was added to re-hash any password not already in bcrypt format.
- **Expired sessions breaking pages.** An expired token used to leave customers on a raw "Missing or invalid Authorization header" message. The API client now clears the session and returns the user through the shared login to the same page.
- **Graded administrator permissions.** A super-administrator tier was introduced for roster management, with the UI hiding the screen and the API enforcing it independently.

8.2 Project Outcomes

The team delivered a working multi-vendor platform with five role-specific front-ends, a REST API of thirteen route modules, eleven data models and an integrated workflow from seller onboarding to delivery and returns. All six repository test scripts passed (107 checks) and the focused authentication and authorization checks matched the behaviour described in the code. Key results in authentication and integration were a single shared login used by all four roles, server-side role enforcement on privileged routes and a tested session-recovery path in the storefront.

8.3 Limitations Identified

The following limitations were identified while verifying the code for this report (October 2026). They are recorded so that they can be addressed, and are not claimed to have been fixed.

- **Unauthenticated read endpoints.** `GET /api/orders` (without a `userId` filter), `GET /api/orders/:id` and `GET /api/delivery-partners` do not require a token. Against the seeded demo data they returned customer names, addresses and phone numbers, and partner documents. These routes should require authentication and filter results by role.
- **Token handling.** The JWT is passed in the URL during the cross-origin hand-off and stored in `localStorage`, where any injected script could read it [13]. Tokens last seven days and there is no refresh-token or revocation list (the post-internship status re-check reduces, but does not remove, this risk).
- **Configuration defaults.** `JWT_SECRET` falls back to a hard-coded development value if the environment variable is missing; CORS and Socket.io accept any origin; and Socket.io rooms are joined from query parameters without verifying a token.

- **No rate limiting** on login, registration or password-reset endpoints, and reset tokens are stored unhashed.
- **Testing scope.** Prepaid (Razorpay) checkout was not exercised, there are no front-end automated tests and the backend scripts require a running server rather than an isolated test database.

CHAPTER 9
LEARNING OUTCOMES, CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Learning Outcomes

- Designing token-based authentication end to end: credential validation, bcrypt hashing, JWT signing and verification, expiry, and the difference between authentication (401) and authorization (403).
- Composing Express middleware to enforce roles, ownership and permission tiers on the server, and treating client-side route guards as usability features rather than security controls.
- Integrating several React applications with one backend, including cross-origin session hand-off, safe redirects and graceful recovery from expired sessions.
- Modelling data with Mongoose (schemas, enums, indexes, transforms) and reasoning about state machines for orders.
- Testing APIs systematically with scripted checks, comparing expected and observed status codes, and reporting limitations honestly.
- Working in a six-member, two-team project with Git and GitHub, pull requests and a shared work-allocation plan.

9.2 Conclusion

The internship at TechPower Solutions provided practical experience of building a real multi-role web application with the MERN stack. ShopSphere, developed jointly by Ctrl+Alt+Win and Femme Forces, demonstrates a complete marketplace workflow across customer, seller, administrator and delivery-partner portals. Each member's work — authentication and the shared login, the overall architecture and integration, the administration and seller portals, and testing of the customer and delivery flows — contributed to one integrated platform. Verifying the system for this report also showed where its security can be strengthened, which forms the basis of the enhancements below.

9.3 Future Enhancements

- Require authentication and role-based filtering on the order and delivery-partner read endpoints identified in Section 8.3.
- Move session tokens to secure, HTTP-only cookies or a short-lived access token with a refresh token, and replace the URL hand-off with a one-time code exchange.
- Fail start-up when `JWT_SECRET` is missing, restrict CORS and Socket.io origins, and authenticate socket connections with the JWT.
- Add rate limiting and lockout on authentication endpoints, hash password-reset tokens and optionally reintroduce e-mail verification or two-factor authentication for administrators.
- Add automated front-end and isolated backend tests (for example with an in-memory MongoDB) to continuous integration, and exercise the Razorpay test-mode flow.
- Deploy the portals behind a single domain so that sessions can be shared without cross-origin hand-off.

REFERENCES

[1] Ctrl+Alt+Win and Femme Forces, “ShopSphere — Multi-Vendor E-Commerce Platform,” GitHub repository. https://github.com/reyhaan07/E-Commerce (main branch, commit 8f85bbd; accessed 21 Oct. 2026).

[2] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015. https://www.rfc-editor.org/rfc/rfc7519

[3] Auth0, “jsonwebtoken” library documentation. https://github.com/auth0/node-jsonwebtoken

[4] D. Wirtz, “bcrypt.js” library documentation. https://github.com/dcodeIO/bcrypt.js

[5] OpenJS Foundation, “Express 5.x API Reference.” https://expressjs.com/en/5x/api.html

[6] Mongoose, “Mongoose Documentation.” https://mongoosejs.com/docs/

[7] MongoDB, Inc., “MongoDB Manual.” https://www.mongodb.com/docs/manual/

[8] Meta Open Source, “React Documentation.” https://react.dev/

[9] Vite, “Vite Guide.” https://vite.dev/guide/

[10] Socket.IO, “Socket.IO v4 Documentation.” https://socket.io/docs/v4/

[11] Razorpay, “Standard Checkout — Verify Payment Signature.” https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/

[12] OWASP Foundation, “Authentication Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Authentication*Cheat*Sheet.html

[13] OWASP Foundation, “HTML5 Security Cheat Sheet — Local Storage.” https://cheatsheetseries.owasp.org/cheatsheets/HTML5*Security*Cheat_Sheet.html

[14] Microsoft, “Playwright Documentation” (used to capture the screenshots). https://playwright.dev/